OCL

Ana Funes
Métodos Formales - Ingeniería en Informática

¿Qué es OCL?

- OCL = **O**bject **C**onstraint **L**anguage
- Lenguaje de especificación formal usado para describir expresiones sobre modelos UML.
  - Invariantes del sistema modelado
  - Consultas sobre objetos definidos en el modelo
  - …
- Es un lenguaje de especificación puro
  - Las expresiones OCL no tienen efectos colaterales
  - Los detalles de implementación quedan fuera de su alcance.
- No es un lenguaje de programación
- Es un lenguaje tipado, por lo que todas las expresiones en OCL tienen un tipo.

¿Dónde usar OCL?

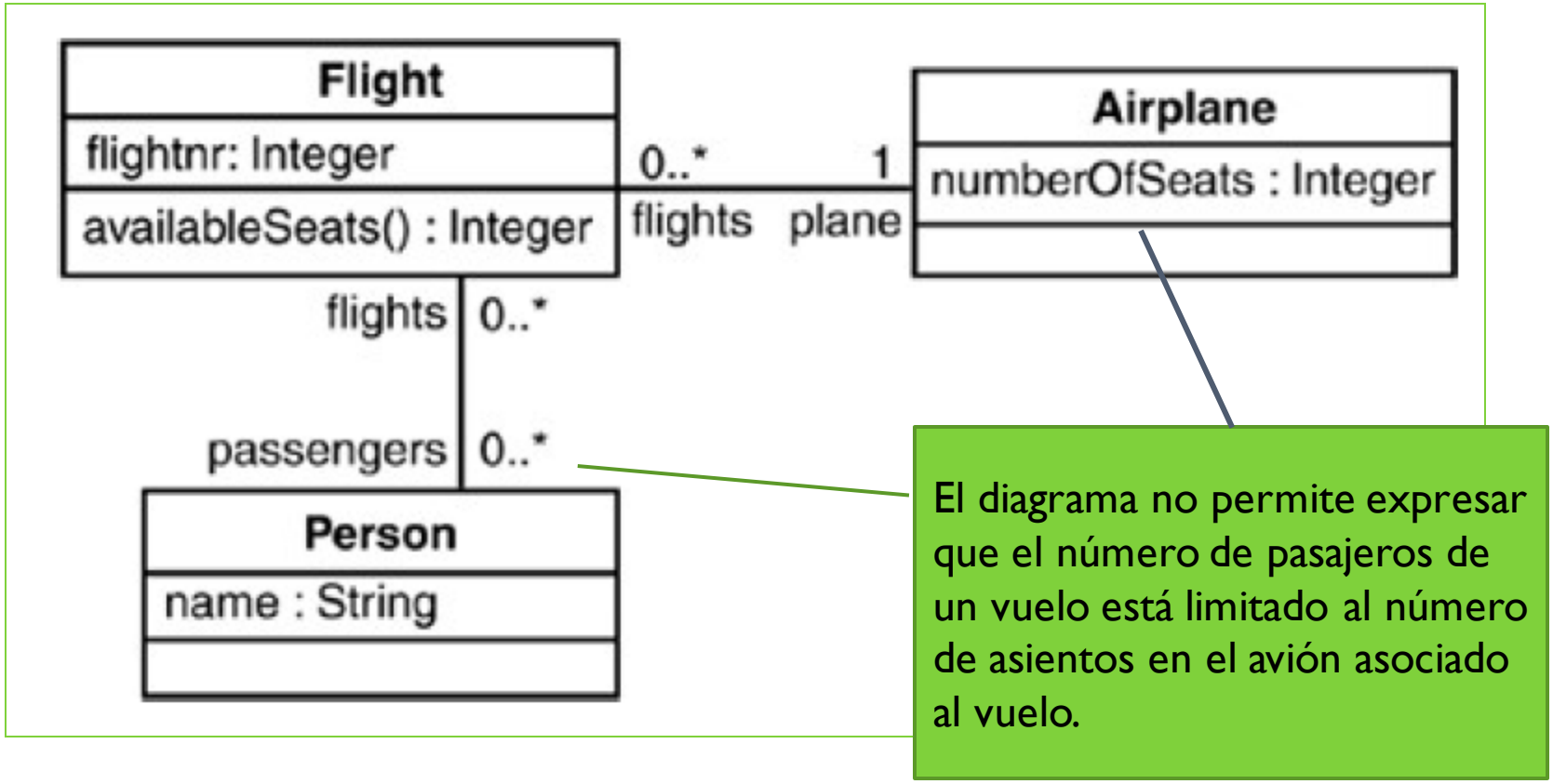
- Como un lenguaje de consulta
- Especificar invariantes sobre clases y tipos en un diagrama de clases
- Especificar invariantes de tipos en Estereotipos
- Describir pre y post condiciones de operaciones y métodos
- Describir condiciones de guarda en máquinas de estado
- Especificar reglas de derivación para atributos
- …

OCL ha sido usado para definir parte de la semántica de UML

¿Por qué OCL?

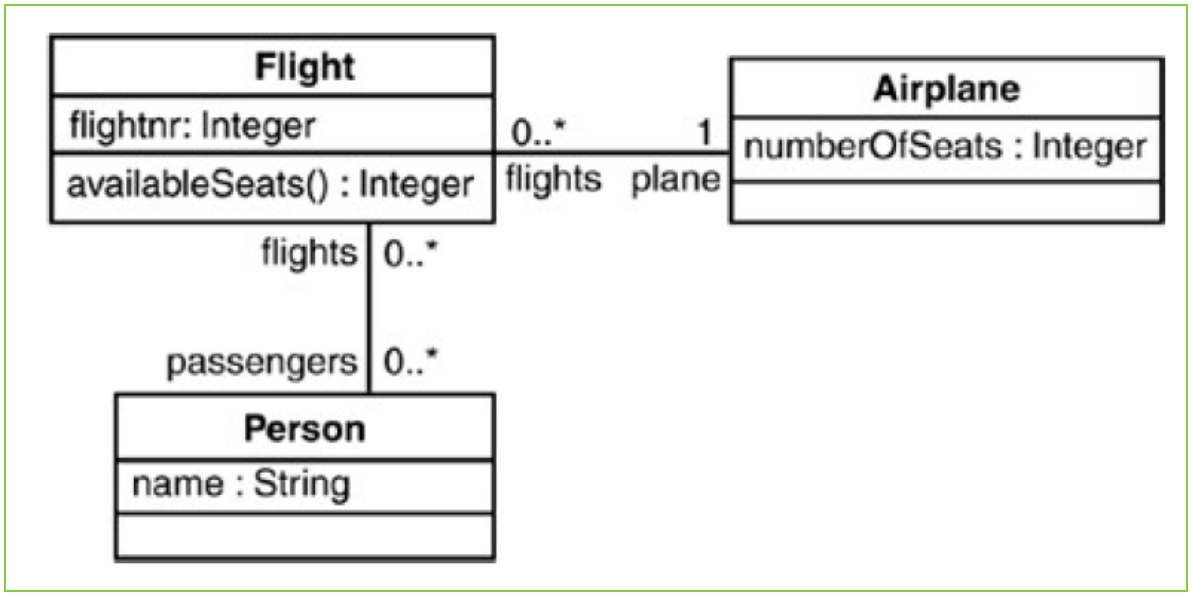
- Los diagramas UML no tienen la suficiente capacidad expresiva para expresar muchos aspectos del sistema modelado.
- Muchas restricciones no pueden ser modeladas en UML.
- Soluciones:
  - Lenguaje natural → problemas de ambigüedades
  - Notación formal tradicional (pe. Alloy, RLS, VDM, Z, etc.) → difíciles de aplicar
  - Solución alternativa: usar OCL →
    - OCL y UML se encuentran naturalmente integrados.
    - Lo mejor de ambos mundos (formalidad y facilidad de comprensión).

¿Por qué OCL? - Ejemplo



¿Por qué OCL? - Ejemplo

Se refiere a la instancia contextual; en este ejemplo, a una instacia de Flight




```
context Flight
inv: self.passengers->size()<= self.plane.numberOfSeats

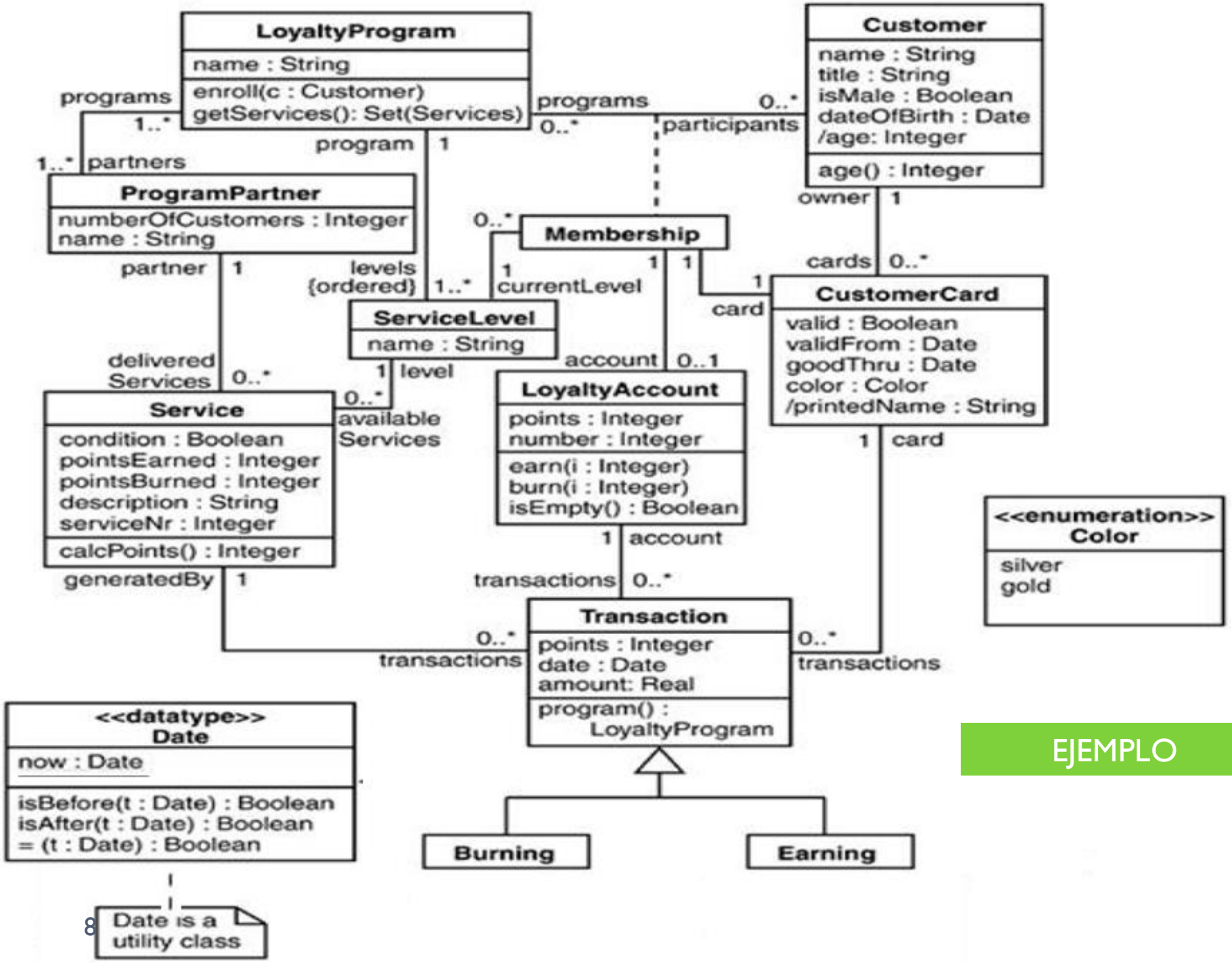
context Flight
inv: passengers->size()<= plane.numberOfSeats

context f:Flight
inv: f.passengers->size()<= f.plane.numberOfSeats
```

Invariante válido para cualquier instancia de Flight

Completando Diagramas de Clase UML con expresiones OCL

- Podemos definir:
  - Valores iniciales de atributos
  - Reglas de derivación (valores de atributos derivados)
  - Cuerpo de operaciones de consulta
  - Invariantes
  - Pre y post condiciones de operaciones


LoyaltyProgram
name : String
enroll(c : Customer)
getServices(): Set(Services)
programs
1..*
1..*
partners
ProgramPartner
numberOfCustomers : Integer
name : String
partner
1
delivered Services
0..*
Service
condition : Boolean
pointsEarned : Integer
pointsBurned : Integer
description : String
serviceNr : Integer
calcPoints() : Integer
generatedBy
1
program
1
levels {ordered}
1..*
ServiceLevel
name : String
1
level
0..*
available Services
programs
0..*
participants
0..*
Customer
name : String
title : String
isMale : Boolean
dateOfBirth : Date
/age: Integer
age() : Integer
owner
1
cards
0..*
Membership
0..*
1
currentLevel
1
1
1
card
CustomerCard
valid : Boolean
validFrom : Date
goodThru : Date
color : Color
/printedName : String
1
card
account
0..1
LoyaltyAccount
points : Integer
number : Integer
earn(i : Integer)
burn(i : Integer)
isEmpty() : Boolean
1
account
transactions
0..*
Transaction
points : Integer
date : Date
amount: Real
program() : LoyaltyProgram
0..*
transactions
0..*
transactions
Burning
Earning
<<enumeration>>
Color
silver
gold
<<datatype>>
Date
now : Date
isBefore(t : Date) : Boolean
isAfter(t : Date) : Boolean
= (t : Date) : Boolean
Date is a utility class

EJEMPLO

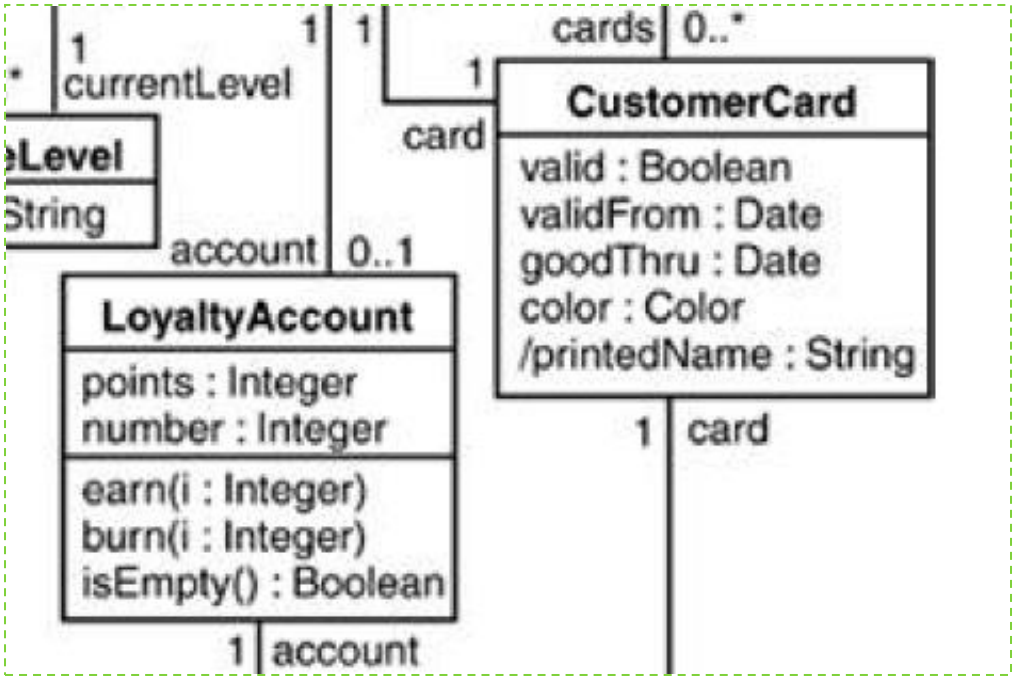
Valores Iniciales de Atributos

- Sintaxis:

```
context Clase::atributo
init: -- expresión OCL representando
      -- el valor inicial
```

Valores Iniciales de Atributos

EJEMPLO




```
context LoyaltyAccount::points
init: 0

context CustomerCard::valid
init: true
```

Reglas de Derivación

- Se debe completar el modelo dando reglas para calcular los valores de los atributos que han sido declarados como atributos derivados.

- Sintaxis:

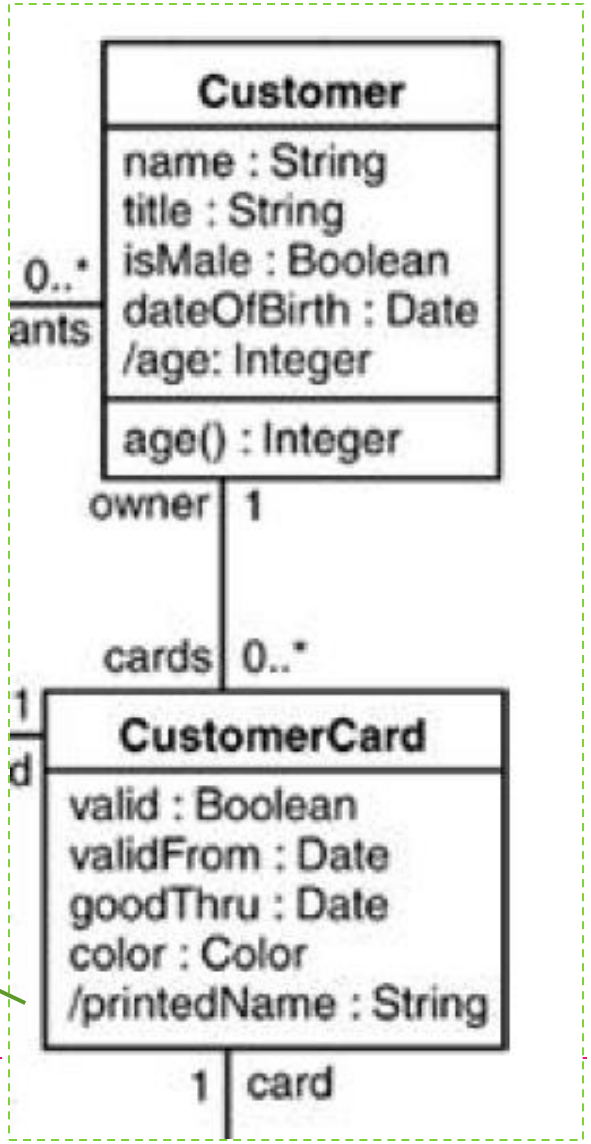
```
context Clase::atributoDerivado
derive: -- regla de derivacion (expresión OCL)
```

Reglas de Derivación

EJEMPLO

```
context CustomerCard::printedName
derive: owner.title.concat(' ').
        concat(owner.name)
```

El atributo derivado "printedName" de la clase "CustomerCard" se obtiene de la concatenación del título del Cliente poseedor de la tarjeta con su nombre, separados por un espacio en blanco.



Cuerpo de Operaciones de Consulta

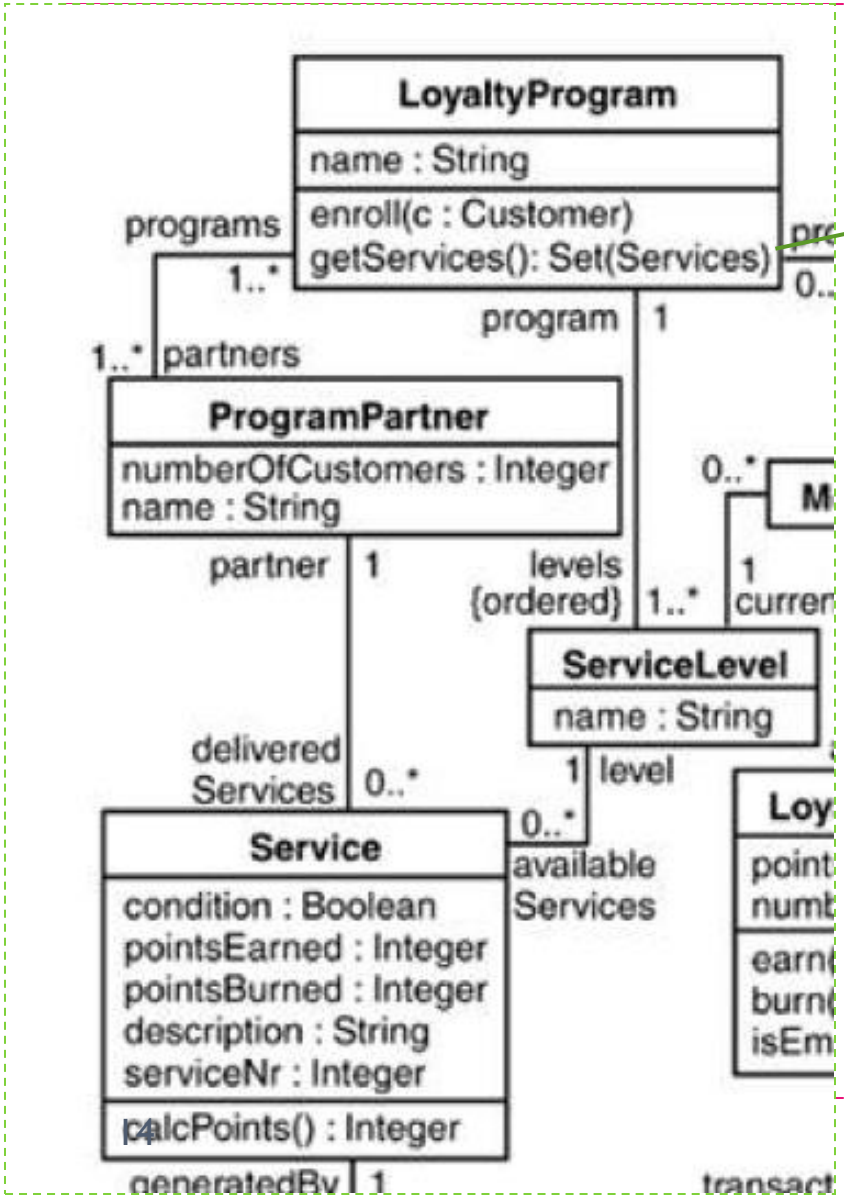
- Las operaciones de consulta son operaciones que no cambian el estado del sistema, simplemente retornan un valor o conjunto de valores.

- Sintaxis:

```
context Clase::nombreOperacion(param1:
        Tipo1, ... ): TipoResultado
body: -- expresión OCL
```

Cuerpo de Operaciones de Consulta

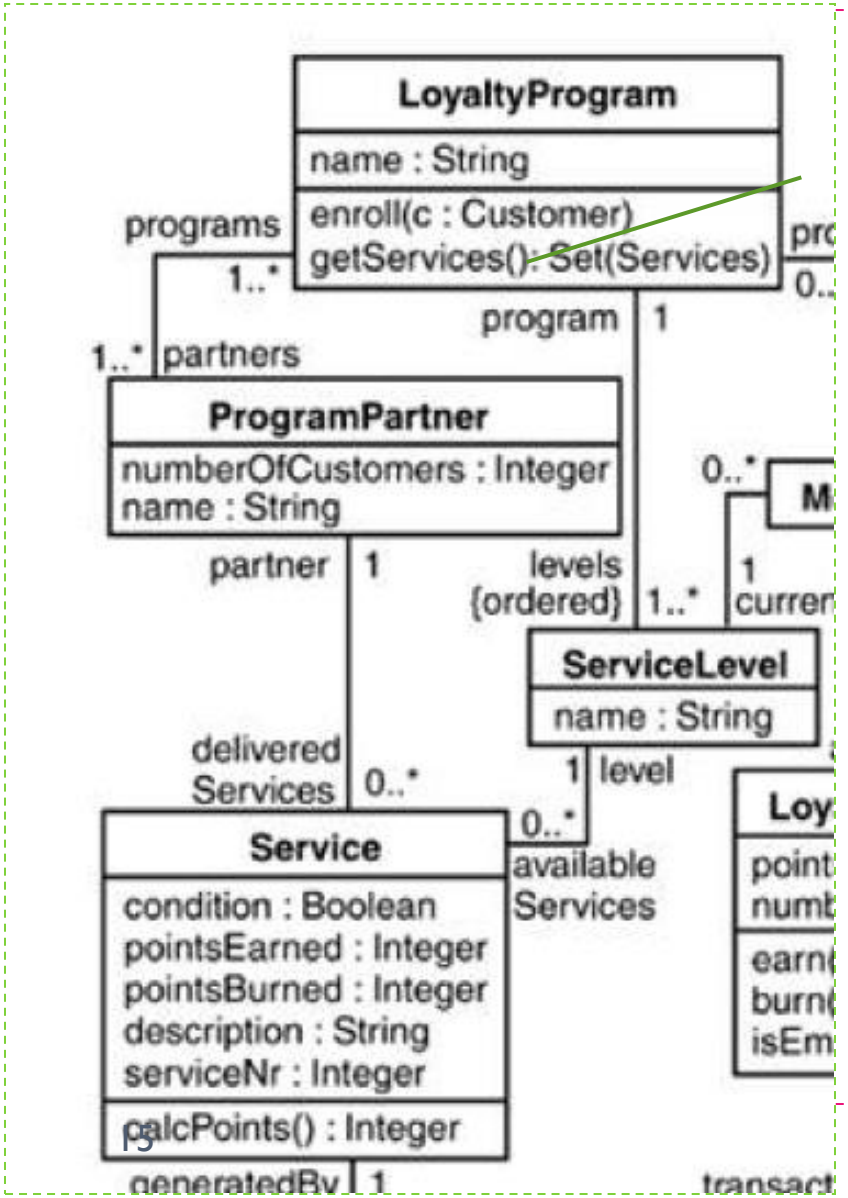
EJEMPLO



La operación `getServices` retorna todos los servicios entregados por todos los socios del programa (`ProgramPartner`) en esta instancia de `LoyaltyProgram`

```
Context LoyaltyProgram::getServices():
        Set(Service)
body: partners.deliveredServices
      ->asSet()
```

Cuerpo de Operaciones de Consulta



EJEMPLO

La operación `getServices` podría, por ej., tomar como parámetro una instancia de `ProgramPartner` y retornar, si se trata de un socio del programa (un `ProgramaPartner`), los servicios entregados (`deliveredServices`) por la instancia parámetro, sino retornar el conjunto vacío.

```
context
  LoyaltyProgram::getServices(
  pp:ProgramPartner):Set(Service)
body: if partners->includes(pp)
        then pp.deliveredServices
        else Set{}
      endif
```

Invariantes

- Un/una *invariante* es una expresión booleana que establece una condición que debe cumplirse para todas las instancias del tipo sobre el cual se ha definido y en todos los estados consistentes del sistema.

- La expresión debe ser verdadera para cada instancia del clasificador. Solo cuando una instancia está ejecutando una operación, el invariante no necesita evaluar a verdadero. Es decir, mientras el sistema está ejecutando una operación, podría no estar en un estado consistente; sin embargo, cuando la ejecución de la operación ha finalizado, el invariante debe mantenerse.

- Sintaxis:

```
context contexto
inv NombreOpcionalInvariante: -- invariante en OCL
```

Invariantes

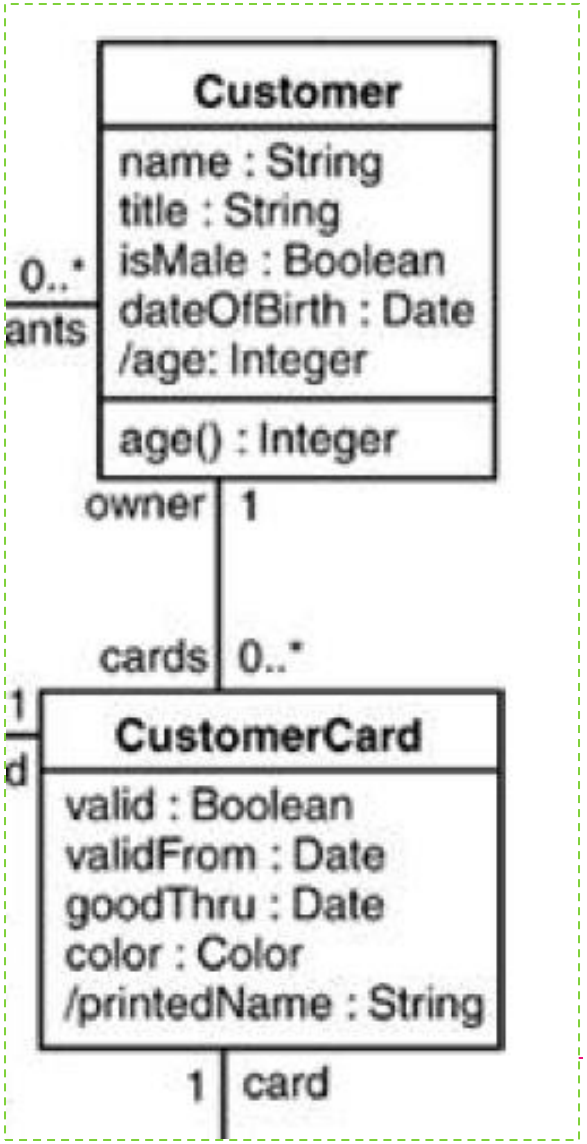
EJEMPLOS

- *Todos los clientes que ingresan en algún programa de fidelidad deben ser mayores de edad (edad >= 18):*

- **context** Customer
  inv ofAge: age >= 18

- *Todo poseedor de una tarjeta debe ser mayor de edad:*

- **context** CustomerCard
  inv ofAge: owner.age >= 18



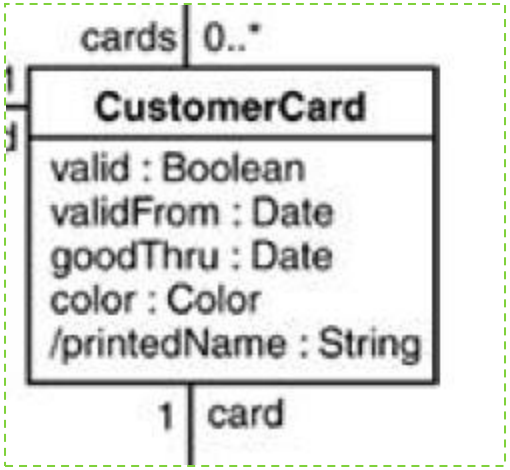
Invariantes

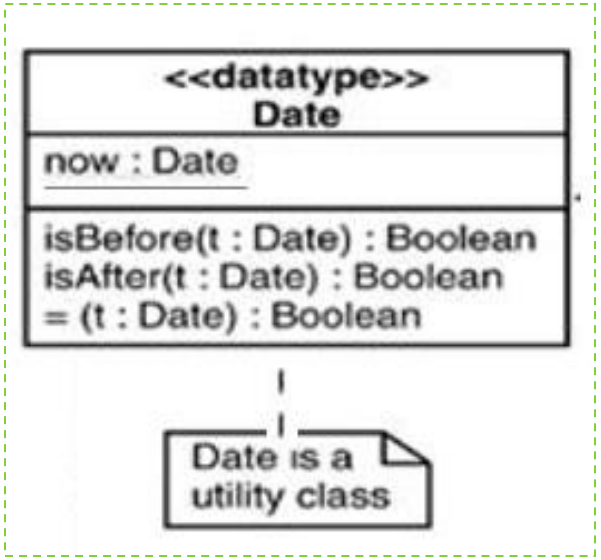
EJEMPLO

- *La fecha de validez desde* (`validFrom`) *debe ser menor que la fecha hasta* (`goodThru`) *en una tarjeta del cliente* (`CustomerCard`):

```
context CustomerCard
inv checkDates:
validFrom.isBefore(goodThru)
```

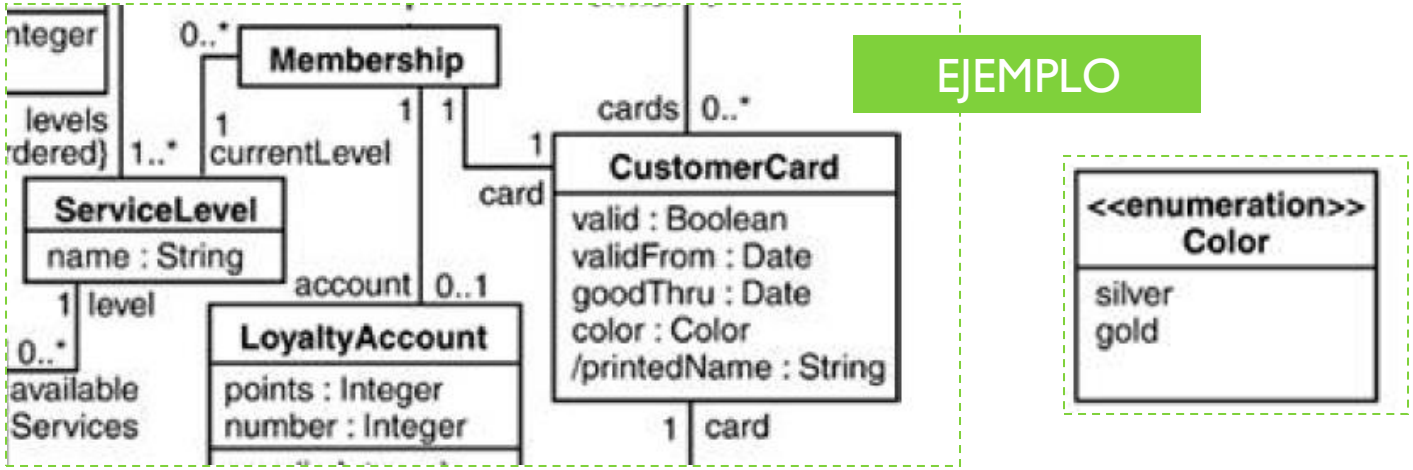
Notar que el invariante usa la operación `isBefore`, que compara dos fechas, de la clase utilitaria `Date`.





Navegación desde una Clase Asociación

- *El nivel de servicio de una membrecía de un cliente debe coincidir con el color de su tarjeta*



- **context** Membership

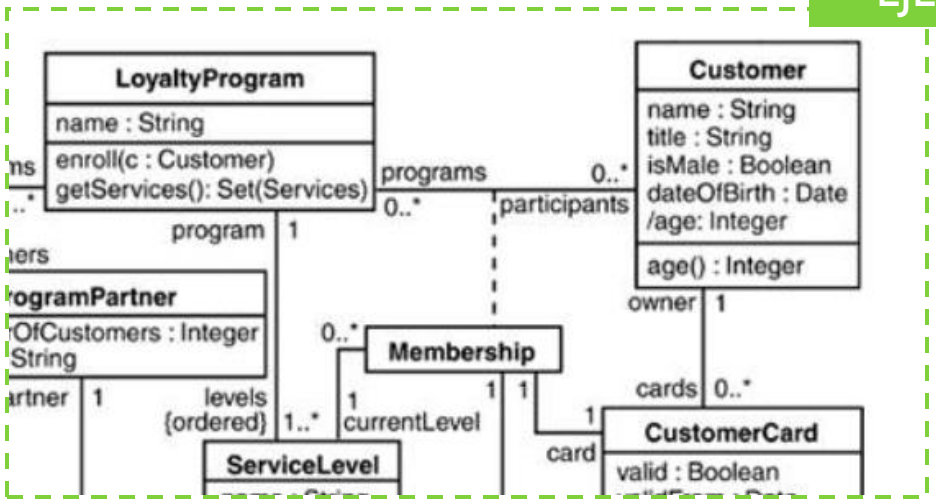
  inv levelAndColor: currentLevel.name = 'Silver' implies card.color = Color::silver and
  currentLevel.name = 'Gold' implies
  card.color = Color::gold

- Notar que el invariante hace referencia a los valores del **tipo enumerado** Color y que Membership es **una clase asociación**

Navegación desde una Clase Asociación

- Usando la **clase asociación** como contexto, se puede navegar a las instancias de las clases en los extremos de la asociación a la que pertenece la clase asociación, utilizando sus nombres de rol.
- Recordar que la navegación desde una clase asociación a uno de los extremos de la asociación siempre entregará exactamente un objeto (Esto resulta de la definición de clase asociación.)

EJEMPLO




```
context Membership
inv correctCard: participants.cards->includes(self.card)
```

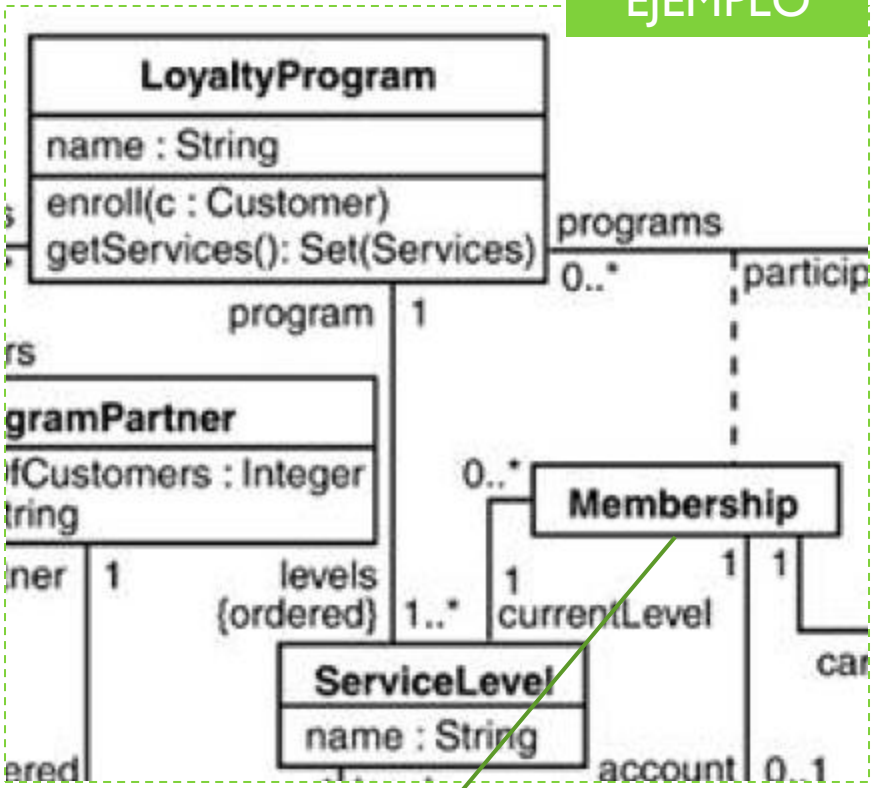
Navegación hacia una Clase Asociación

- *En un programa de fidelidad* (`LoyaltyProgram`) *el nivel de servicio de cada membrecía debe ser un nivel de servicio conocido por el programa*:

```
context LoyaltyProgram
inv: levels-> includesAll(
     Membership.currentLevel)
```

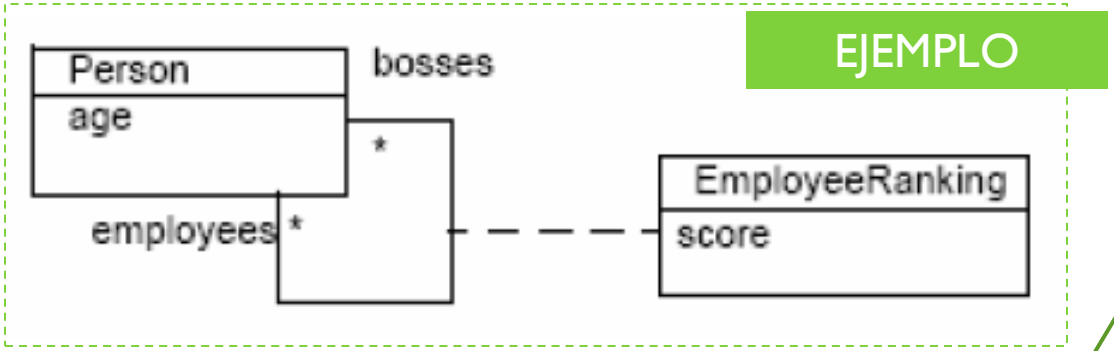
Notar que el invariante es verdadero cuando `levels` incluye todos los elementos de `Membership.currentLevel`

EJEMPLO



Como las clases asociación no tienen nombre de rol, se usa el nombre de la clase asociación para navegarla.

Navegación hacia una Clase Asociación



Para distinguir la dirección de navegación, el **rolename** de la dirección **hacia** la que queremos navegar se agrega al nombre de la clase de asociación, encerrado entre corchetes.

```
context Person
inv: self.EmployeeRanking.score->sum() > 0 -- INVALIDO!
```

```
-- sumatoria de puntajes de los rankings de una persona con
-- cada uno de sus jefes
context Person
inv: self.EmployeeRanking[bosses].score->sum() > 0
```

```
-- sumatoria de puntajes de los rankings de los empleados
-- de una persona
context Person
inv: self.EmployeeRanking[employees].score->sum() > 0
```

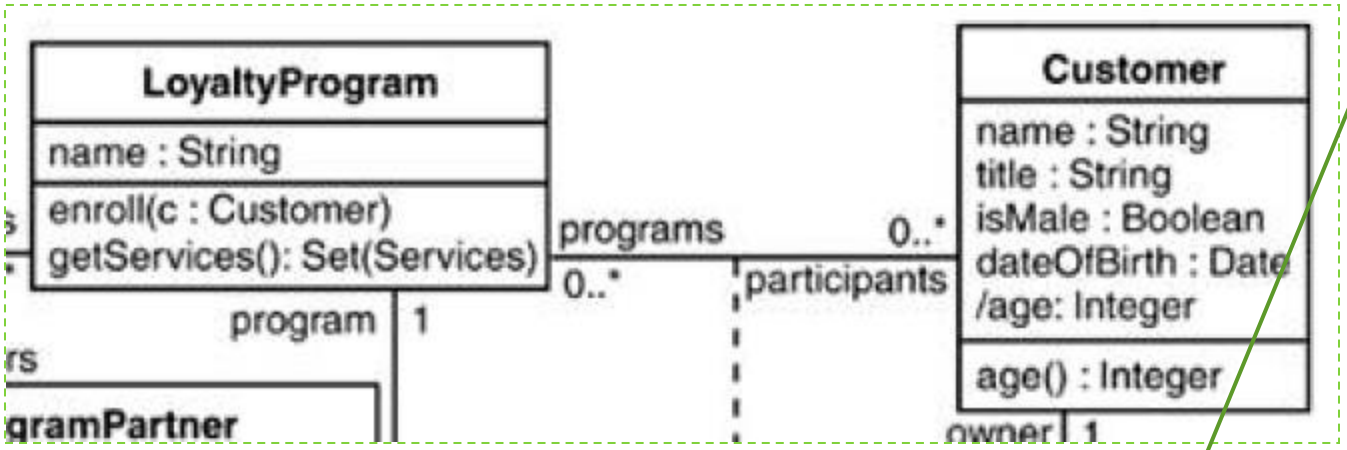
Pre y Post Condiciones de Operaciones

- El principio detrás del uso de pre y postcondciones en la definición de operaciones es referido como **Diseño por Contrato** (Design by Contract).
- **Precondición**: expresión booleana que debe ser verdadera al momento que la operación *comienza* su ejecución.
- **Postcondición** : expresión booleana que debe ser verdadera en el momento que la operación *finaliza* su ejecución.
- Dado que las pre y postcondiciones no especifican cómo se debe implementar el cuerpo de una operación, resultan una manera efectiva de definir las interfaces del sistema.

Pre y Post Condiciones de Operaciones

EJEMPLO

- *Un cliente es inscripto (*`enroll`*) en un programa de fidelidad (*`LoyaltyProgram`*) bajo la condición de que su nombre (*`name`*) no sea vacío. La postcondición de la operación* `enroll` *asegura que el nuevo cliente se haya incluido en la lista de clientes que pertenecen al programa de fidelidad:*



Hace referencia al valor al inicio de la operación.

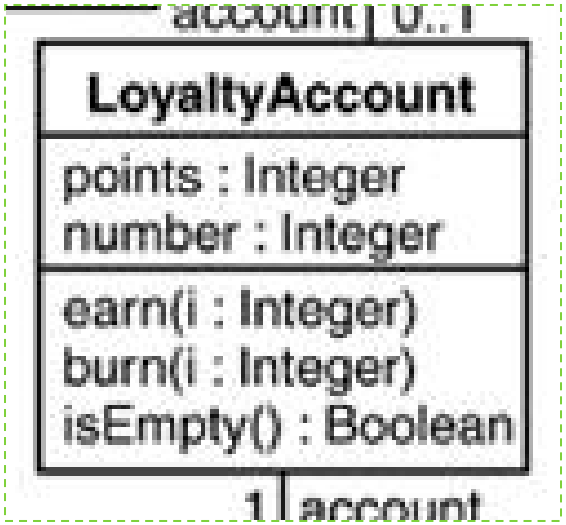
```
context LoyaltyProgram::enroll(c: Customer)
pre: c.name <> ''
post: participants = participants@pre ->including(c)
```

Agrega c a la colección de participants@pre

Pre y Post Condiciones de Operaciones – Uso de **`result`**

EJEMPLO

- La operación `isEmpty()` de `LoyaltyAccount` devuelve verdadero cuando el número de puntos en la cuenta de fidelidad es cero.




```
context LoyaltyAccount::isEmpty(): Boolean
pre : -- ninguna
post: result = (points = 0)
```

`result` es una palabra clave. Contiene el valor de retorno de la operación.

Expresiones **if-then-else**

- Una expresión **`if-then-else`** devuelve como resultado una de dos expresiones alternativas según el valor evaluado de una condición.
- Sintaxis:

```
if OCLBoolExpresion then
    OCLExpresion1
else
    OCLExpresion2
endif
```

- Tanto la expresión del **`then`** como la expresión del **`else`** son obligatorias y ambas deben ser del mismo tipo.

Expresiones **let**

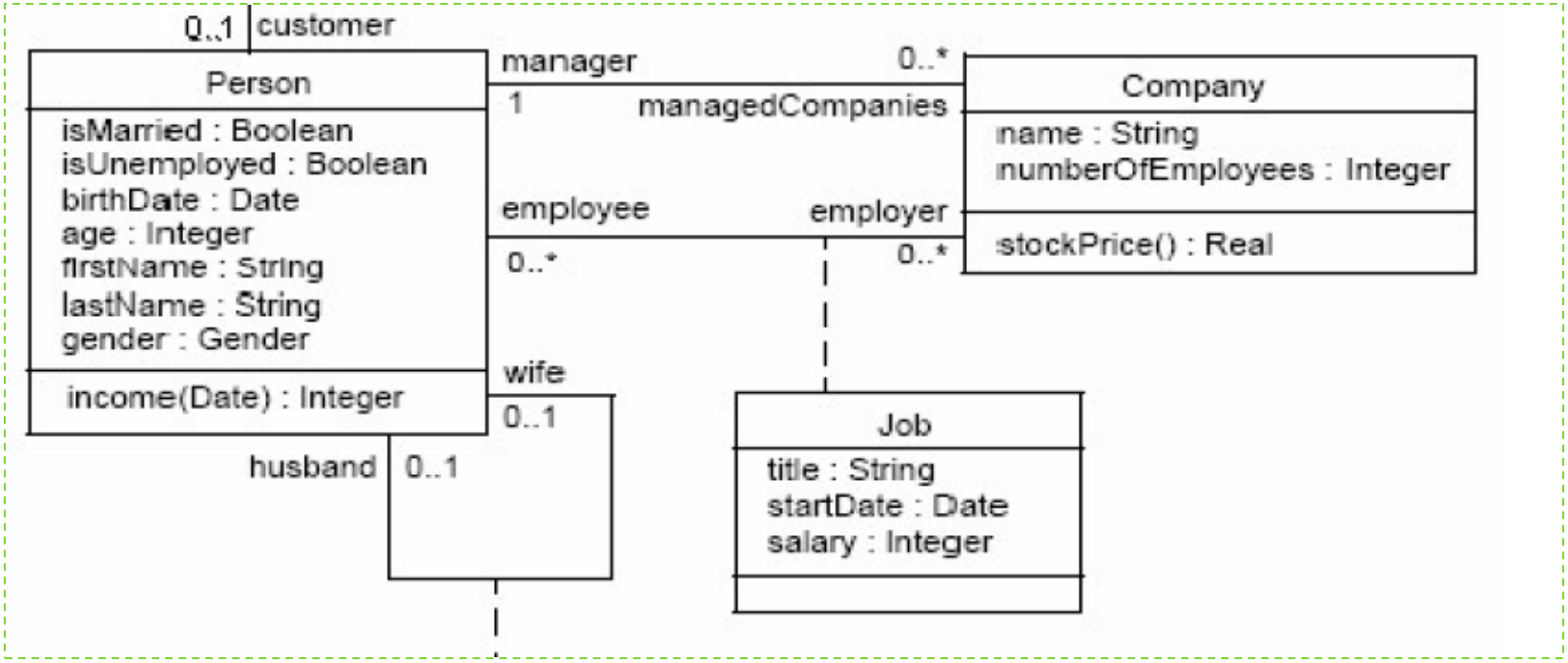
- Permite definir una `variable` que representa una expresión que se puede usar en una restricción múltiples veces.

```
let variable: Tipo = OCLexpresion1
in OCLexpresion2
```

- `variable` no puede cambiar su valor y es visible solo en `OCLexpresion2`.

Expresiones **if-then-else** y expresiones **let**

EJEMPLO




```
context Person
inv: let income: Integer = self.Job.salary->sum()
     in if isUnemployed then income = 0
        else income >= 100 endif
```

Tipos en OCL

- Tipos del Modelo UML
  - Clasificadores (tipos, clases, interfaces)
  - Enumeraciones
- Tipos Simples
- Tipos Colección
- Tuplas

Tipos en OCL:
Tipos del Modelo UML

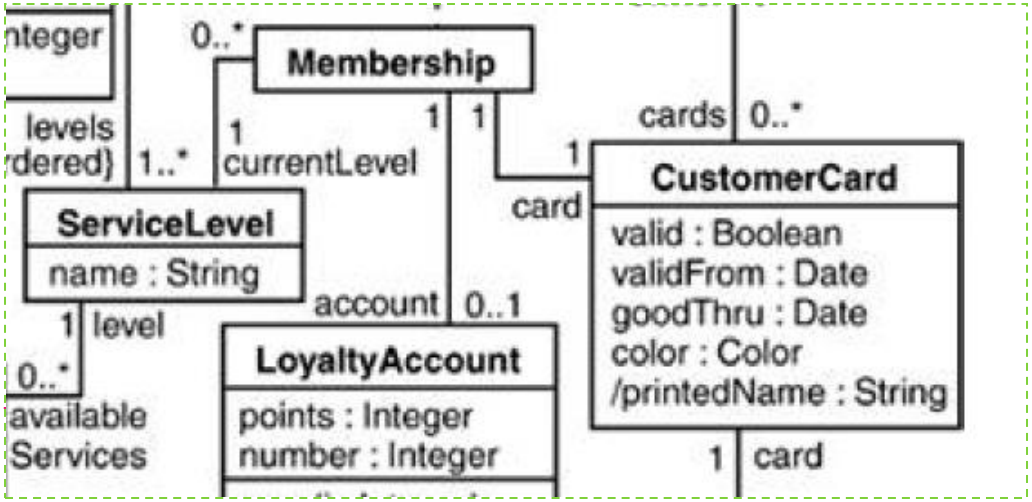
- Cada expresión OCL está escrita en el contexto de un modelo UML el cual cuenta con una serie de clasificadores (tipos, clases, interfaces), de características (atributos, operaciones y finales de asociaciones), y generalizaciones.

- Todos los clasificadores del modelo UML son tipos en OCL.

Tipos en OCL: Tipos Enumerados

- Las enumeraciones son tipos de datos en UML y tienen un nombre, como cualquier otro clasificador.
- Una enumeración define una cantidad de literales de enumeración que son los valores posibles del tipo enumerado.

- ```
  context Membership
  inv: currentLevel.name = 'Silver' implies
  card.color = Color::silver and
  currentLevel.name = 'Gold' implies
  card.color = Color::gold
  ```


  ```

Tipos en OCL: Tipos Simples

Para mayor información ver sec. 11.5 de la especificación de OMG

Tipo	Valores	Operaciones
Real	1.5, 6.33, -4.0, …	+, -, /, *, -, abs(), floor(), round(), max(.), min(.), >, <, =, <=, >=, <>, toString()
Integer	1, -5, 2, 26524, ...	+, -, /, *, -, div(.), mod(.), max(.), min(.), >, <, =, <=, >=, <>, toString()
String	'To be or not to be...'	+(.), size(), concat(.), substring(.,.), toInteger(), toReal(), toUpperCase(), toLowerCase(), indexOf(.) …
Boolean	true, false	or(.), xor(.), and(.), not(), implies(.).toString()
UnlimitedNatural	0, 1, 2, 42, ..., *	+, -, /, *, -, div(.), mod(.), max(.), min(.), >, <, =, <=, >=, <> toInteger(), toString()
OclInvalid	invalid (error de evaluación)	
OclVoid	null (ausencia de valor)	

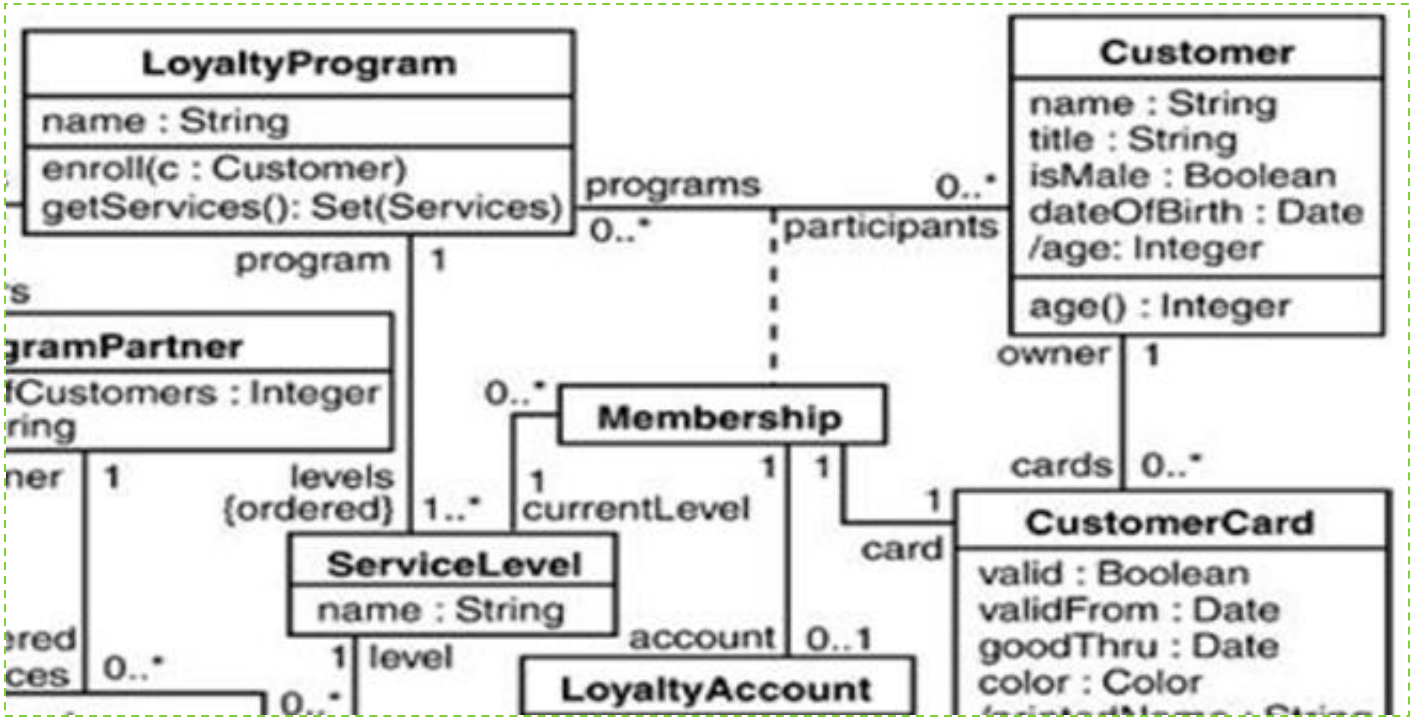
Tipos en OCL: Tipos Colección

Tipo Colección	Ejemplos de Valores
Collection	Tipo abstracto (sin valores concretos)
Set	Set { 1, 2, 5, 88 } Set { 'apple' , 'orange', 'strawberry' } Set { }
OrderedSet	OrderedSet { 'apple' , 'orange', 'strawberry', 'pear' }
Bag	Bag { 1, 3, 4, 3, 5 } Bag { }
Sequence	Sequence { 1, 3, 45, 2, 3 } Sequence { 'ape', 'nut' } Sequence { } ; Sequence { 1..10 }

Tipos en OCL: Tipos Colección

EJEMPLO

- Considerando como contexto `LoyaltyProgram`:
  - `self.participants` → `Set(Customer)`
  - `self.levels` → `OrderedSet(ServiceLevel)`



Tipos en OCL: Operaciones sobre Tipos Colección

Operación	Descripción
`sourceCollection->count(object)`	El número de ocurrencias del objeto en *sourceCollection*.
`sourceCollection->excludes(object)`	Devuelve verdadero si el objeto no es un elemento de *sourceCollection*.
`sourceCollection->excludesAll(collection)`	Devuelve verdadero si todos los elementos en la colección parámetro no están presentes en *sourceCollection*.
`sourceCollection->includes(object)`	Devuelve verdadero si el objeto es un elemento de *sourceCollection*.
`sourceCollection->includesAll(collection)`	Devuelve verdadero si todos los elementos de la colección de parámetros están presentes en la colección actual.
`sourceCollection->isEmpty()`	Devuelve verdadero si la colección no contiene elementos
`sourceCollection->notEmpty()`	Devuelve verdadero si *sourceCollection* contiene uno o más elementos.
`sourceCollection->size()`	Devuelve la cantidad de elementos en *sourceCollection*.
`sourceCollection->sum()`	Devuelve la suma de todos los elementos en *sourceCollection*. Los elementos deben ser de un tipo que soporte la suma (por ej. Real o Integer)

Tipos en OCL: Operaciones sobre Tipos Colección

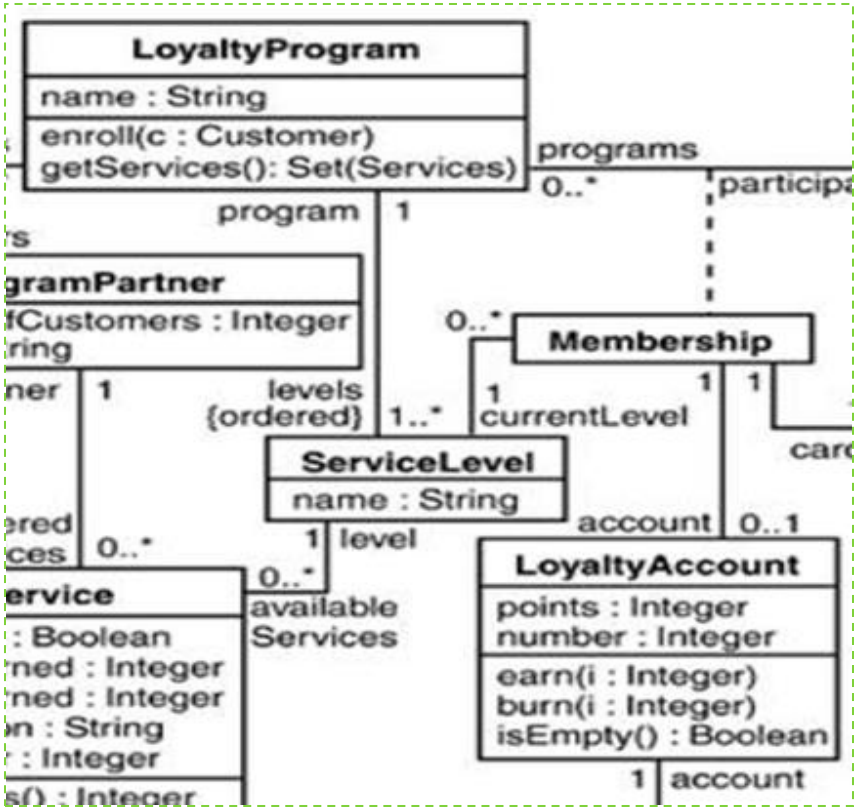
Operación	Descripción
`sourceCollection->any(expr)`	Devuelve un elemento aleatorio de *sourceCollection* para la cual la expresión *expr* es verdadera.
`sourceCollection ->collect(expr)`	Devuelve la colección aplanada de objetos que resulta de la evaluación de *expr* para cada elemento en *sourceCollection*.
`sourceCollection ->collectNested(expr)`	Devuelve una colección anidada de objetos que resulta de la evaluación de *expr* para cada elemento en *sourceCollection*.
`sourceCollection->exists(expr)`	Devuelve verdadero si hay al menos un elemento en la colección de origen para el que *expr* es verdadero.
`sourceCollection->forAll(expr)`	Devuelve verdadero si *expr* es verdadero para todos los elementos en *sourceCollection*.

Tipos en OCL: Operaciones sobre Tipos Colección

Operación	Descripción
`sourceCollection->isUnique(expr)`	Devuelve verdadero si *expr* es un valor único en *sourceCollection*
`sourceCollection->iterate( ... )`	Itera sobre todos los elementos en *sourceCollection*
`sourceCollection->one(expr)`	Devuelve verdadero si existe exactamente un elemento en *sourceCollection* para el cual *expr* es verdadero
`sourceCollection->reject(expr)`	Devuelve una sub colección de *sourceCollection* que contiene todos los elementos para los cuales *expr* es falso
`sourceCollection->select(expr)`	Devuelve una sub colección de *sourceCollection* que contiene todos los elementos para los cuales *expr* es verdadera

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

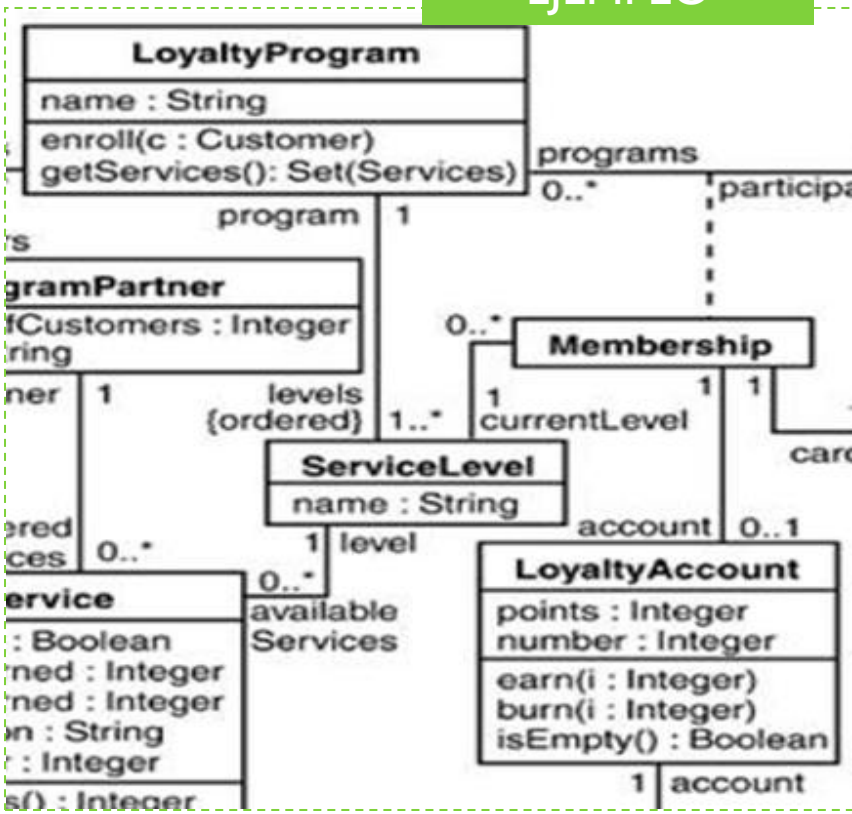


El número de la cuenta de fidelidad (`LoyaltyAccount`) *de una membrecía debe ser único dentro de un programa de fidelidad* (`LoyaltyProgram`).

```
context LoyaltyProgram
inv: self.Membership.account->isUnique(
          acc: LoyaltyAccount | acc.number)
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

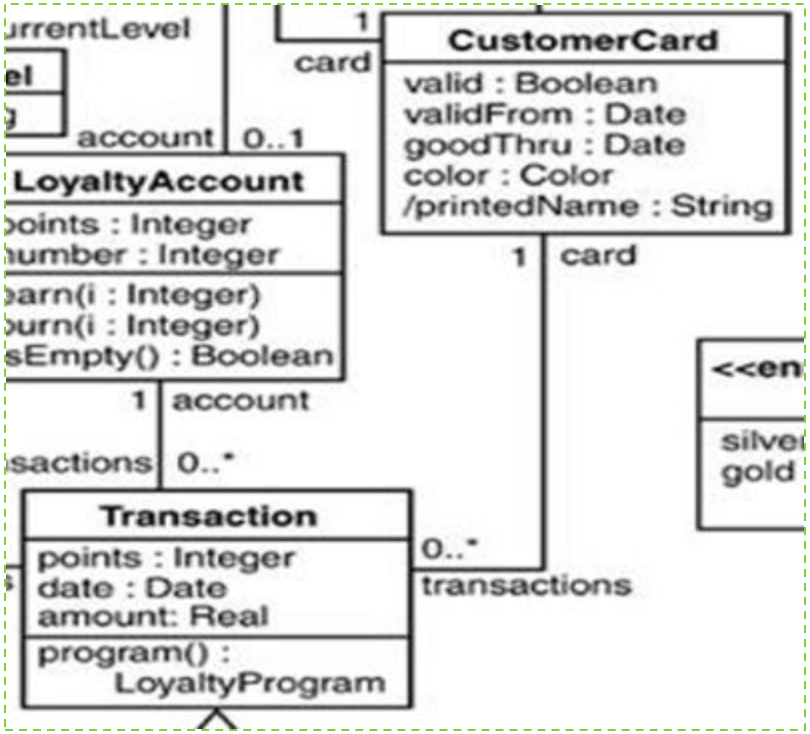


Se define una variable `sortedAccounts` *dentro de una expresión* `let` *que contiene todas las cuentas de fidelidad de un programa de fidelidad ordenadas por número:*

```
context LoyaltyProgram
inv: let sortedAccounts: OrderedSet(LoyaltyAccount)
  = self.Membership.account->sortedBy(number) in …
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO



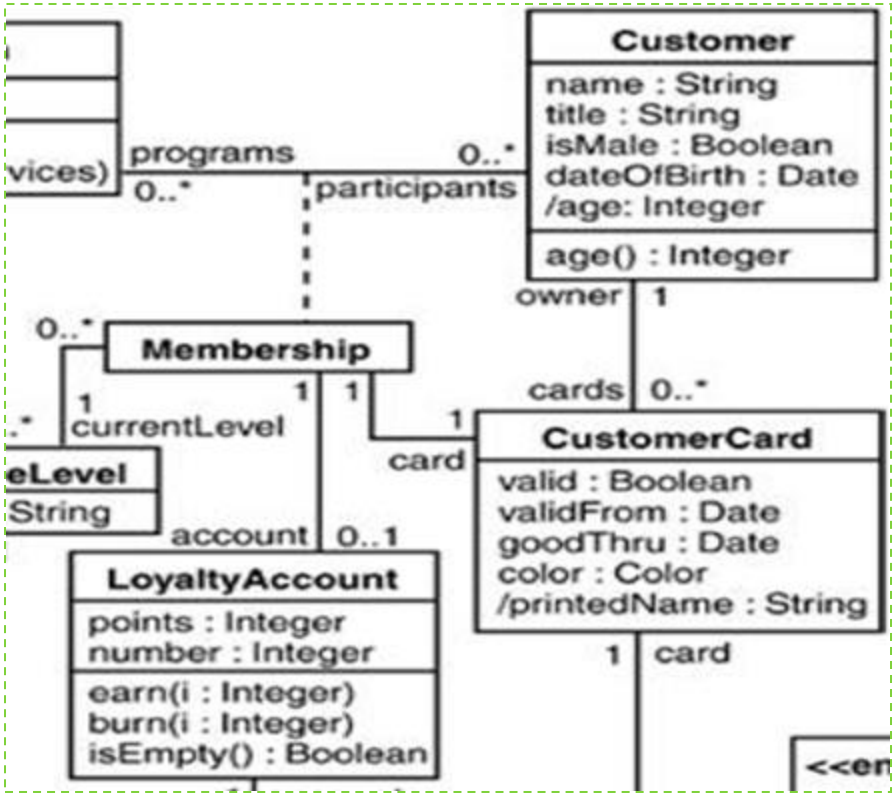
En toda tarjeta de cliente (`CustomerCard`) debe existir al menos una transacción con mas de 100 puntos.

```
context CustomerCard
inv: self.transactions->
     select(points > 100) ->notEmpty()
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

Todo cliente debe tener al menos una cuenta de membrecía que tenga puntos.

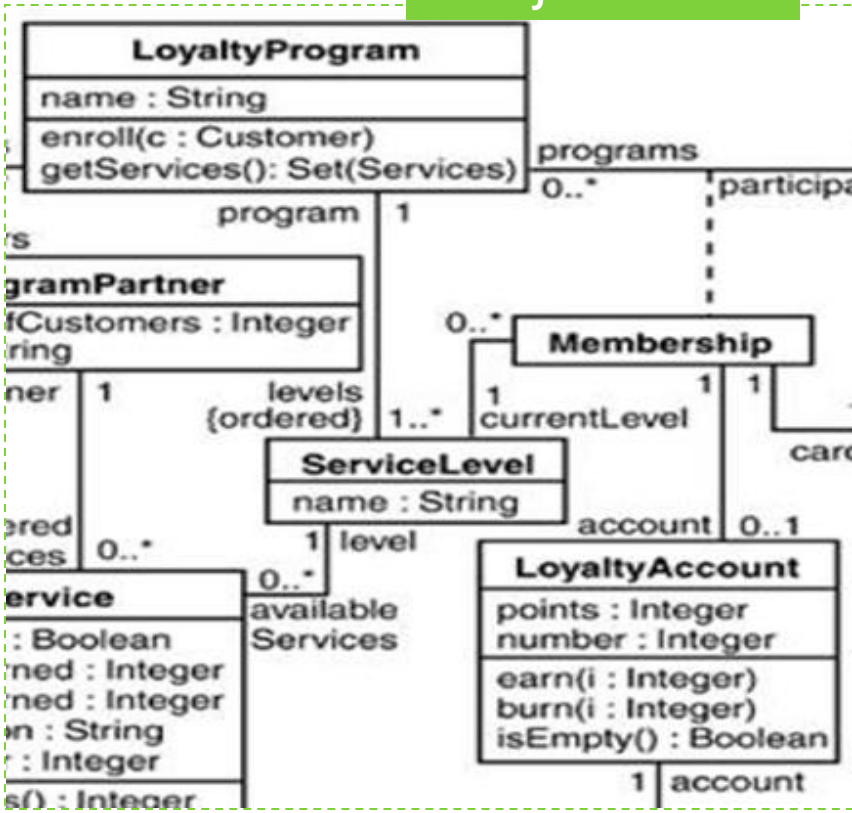



```
context Customer
inv: Membership.account->
     reject(points <= 0)-> notEmpty()
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

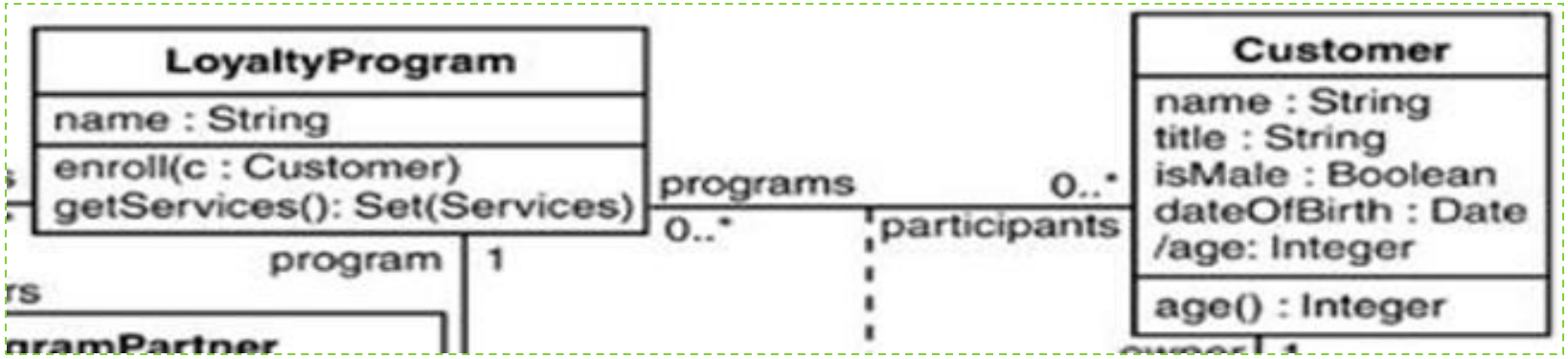
En el contexto de `LoyaltyProgram`*, devuelve alguna cuenta de membrecía de un cliente elegida al azar cuyo número de cuenta sea menor a 10000.*




```
context LoyaltyProgram
(...)
self.Membership.account->any(number < 10000)
(...)
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO



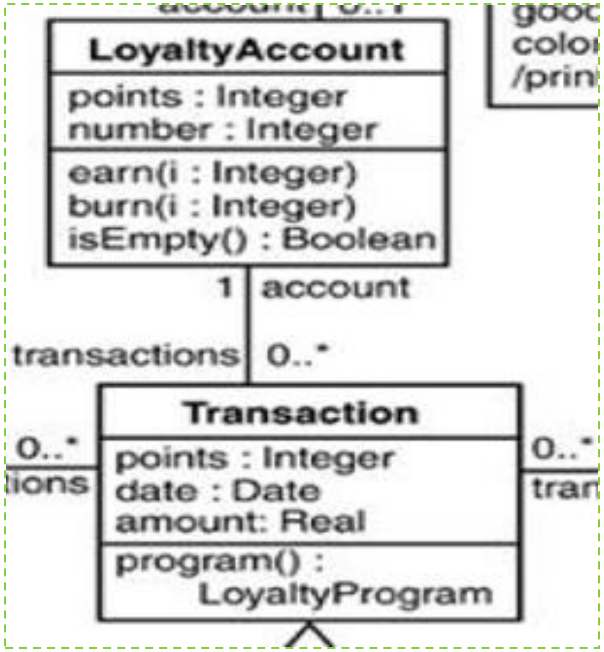
La edad de todos los participantes en un programa de fidelidad es menor o igual a 70.

```
context LoyaltyProgram
inv: participants->forAll(age() <= 70)
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

Si los puntos de una cuenta de fidelidad son mayores a cero, existe al menos una transacción con puntos mayores a cero.

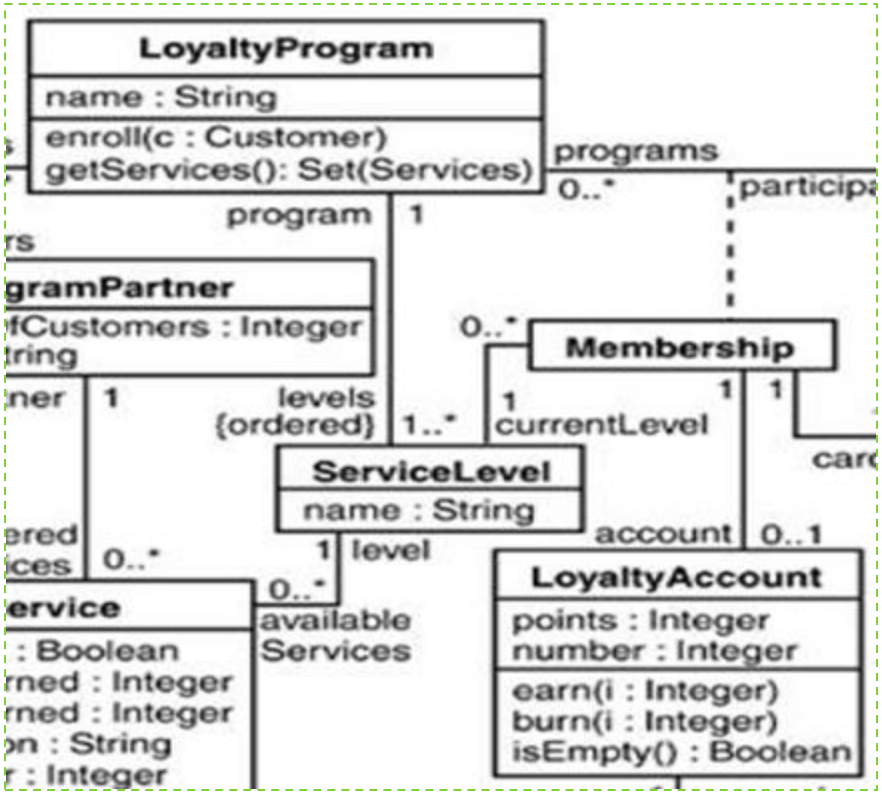



```
context LoyaltyAccount
inv: points > 0 implies transactions ->
     exists(t | t.points > 0)
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

En cada programa de fidelidad puede haber solo una cuenta de membrecía que tenga un número de cuenta inferior a 10000.

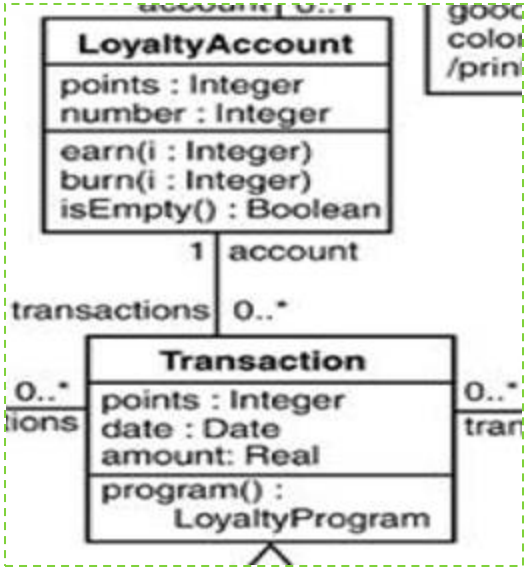



```
context LoyaltyProgram
inv: self.Membership.account->
  one(number < 10000)
```

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLO

```
context LoyaltyAccount
inv: transactions->collect(points)->
  exists(p: Integer | p > 500)
```



≡

```
context LoyaltyAccount
inv: transactions.points->
  exists(p: Integer | p > 500)
```

Tipos en OCL: Operaciones sobre Tipos Colección

- `Collect` devuelve siempre colecciones donde pueden haber elementos repetidos
  - Aplicado a un conjunto o a una bolsa, devuelve una bolsa
  - Aplicado a un conjunto ordenado o a una secuencia, devuelve una secuencia

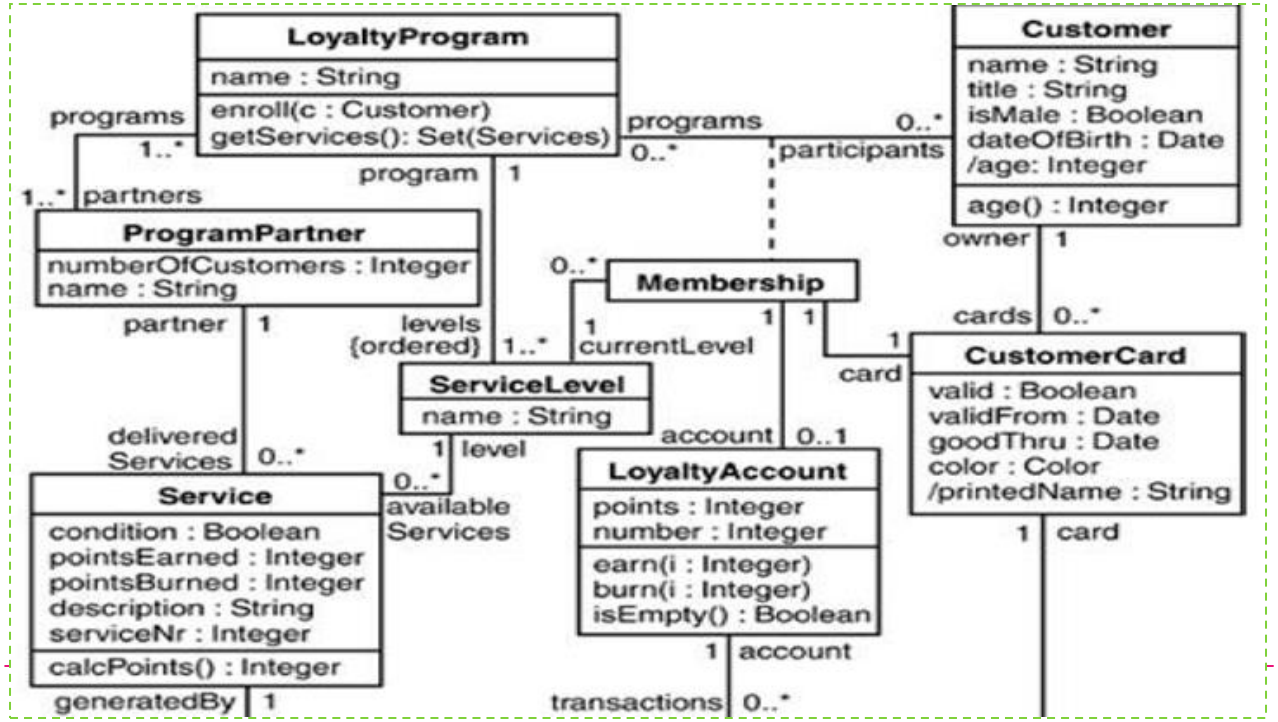
Operadores y Atajos de Navegación

Operador	Si la fuente es un Objeto	Si la fuente es una Colección
.	**Operador** de navegación de objeto. Ejemplos: • `anObject.name` • `aString.indexOf(':')`	**Atajo** de navegación. Ejemplo: `aSet.name` es un atajo a `aSet -> collect(name)`
–>	**Atajo** de navegación de conversión implícita. Ejemplo: `anObject-> union(aSet)` es un atajo de `anObject.asSet()-> union(aSet)`	**Operador** de navegación de colección. Ejemplos: • `aSet ->union(anotherSet)` • `aSet ->collect(name)`

Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLOS

```
context Customer
(...)
self.programs->collect(partners)->
  collectNested(deliveredServices)
(...)
```



Tipos en OCL: Operaciones sobre Tipos Colección

EJEMPLOS

```
collection->iterate(element: Type1;
    acum: Type2 = expression |
  expression-with-element-and-acum)
```

```
-- la suma de los elementos del conjunto de
-- enteros {1,2,3}
Set{1,2,3} -> iterate(
    i: Integer; sum: Integer = 0 | sum + i )
```

Tipos en OCL: Más Operaciones sobre Tipos Colección (i)

Operación	Set	OrderedSet	Bag	Sequence
=	X	X	X	X
<>	X	X	X	X
-	X	X	-	-
append(*object*)	-	X	-	X
asBag()	X	X	X	X
asOrderedSet()	X	X	X	X
asSequence()	X	X	X	X
asSet()	X	X	X	X
at(*index*)	-	X	-	X
excluding(*object*)	X	X	X	X
first()	-	X	-	X

Tipos en OCL: Más Operaciones sobre Tipos Colección (ii)

Operación	Set	OrderedSet	Bag	Sequence
`flatten()`	X	X	X	X
`including(` *object* `)`	X	X	X	X
`indexOf(` *object* `)`	-	X	-	X
`insertAt(` *index*, *object* `)`	-	X	-	X
`intersection(` *collection* `)`	X	-	X	-
`last()`	-	X	-	X
`prepend(` *object* `)`	-	X	-	X
`subOrderedSet(` *lower*, *upper* `)`	-	X	-	-
`subSequence(` *lower*, *upper* `)`	-	-	-	X
`symmetricDifference(`*collection*`)`	X	-	-	-
`union(` *collection* `)`	X	X	X	X

Tipos en OCL: Tipo Tupla (Tuple)

- Consiste de partes nombradas, cada una de tipos posiblemente diferentes.
- Ejemplos de valores del tipo `Tuple(name: String, age: Integer):`

```
Tuple {name: String = 'John',
   age: Integer = 10 }
Tuple {name = 'John', age = 10 }
Tuple {age = 10, name = 'John'}
```

≡

Ejemplos de valores del tipo `Tuple(a: Collection(Integer), b: String, c: String):`

```
Tuple {     a: Collection(Integer) = Set{1,3,4},
            b: String = 'foo',
            c: String = 'bar' }
```

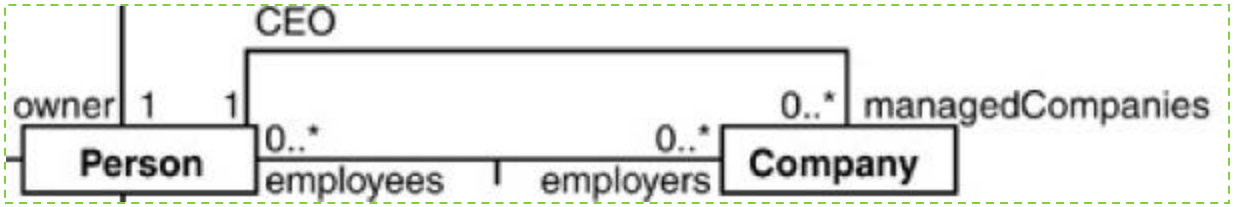
- Se accede a las partes de una tupla por sus nombres, usando la misma notación de puntos que se usa para acceder a los atributos, por ejemplo:

```
Tuple {name: String = 'John', age: Integer = 10}.name = 'John'
```

Tipos en OCL: Tuplas

EJEMPLO

```
context Person
(…)
let statistics: Set(Tuple( company: Company,
                           numEmployees: Integer))=
        managedCompanies -> collect(c | Tuple{
             company: Company = c,
             numEmployees: Integer = c.employees ->
                   size()}) -> asSet()
 in
(…)
```



Herencia y Restricciones en OCL

- Principio de sustitución de Liskov:
  - *Cada vez que se espera una instancia de una clase, siempre puede sustituirse por cualquiera de sus subclases.*
- En consecuencia:

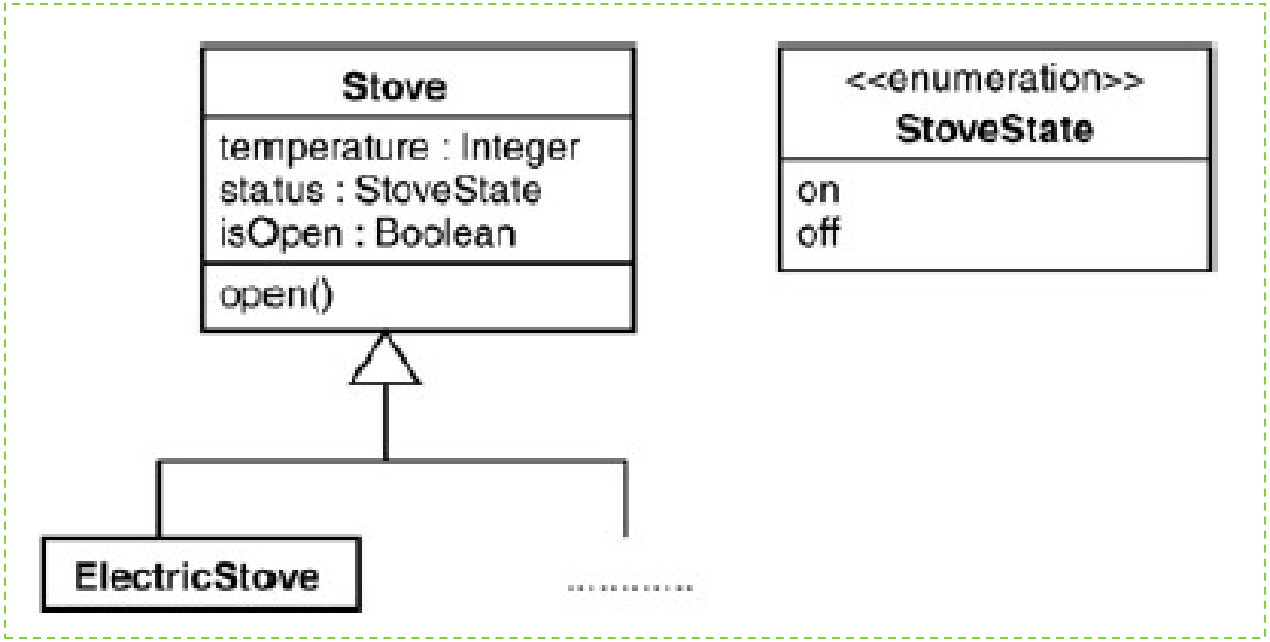
1. Un invariante para una superclase es heredado por sus subclases. Una subclase puede fortalecer al invariante pero nunca debilitarlo.
2. En una redefinición de una operación en una subclase,
   a. una precondición puede debilitarse, nunca fortalecerse.
   b. una postcondición puede fortalecerse, nunca debilitarse.

Herencia y Restricciones en OCL: Invariantes

```
context Stove
inv: temperature <= 200

context ElectricStove
inv: temperature <= 300
```

Un invariante de la subclase **puede fortalecer** al **invariante** de la superclase pero **nunca debilitarlo**.



Herencia y Restricciones en OCL: Invariantes

Si x fortalce y entonces x $\Rightarrow$ y

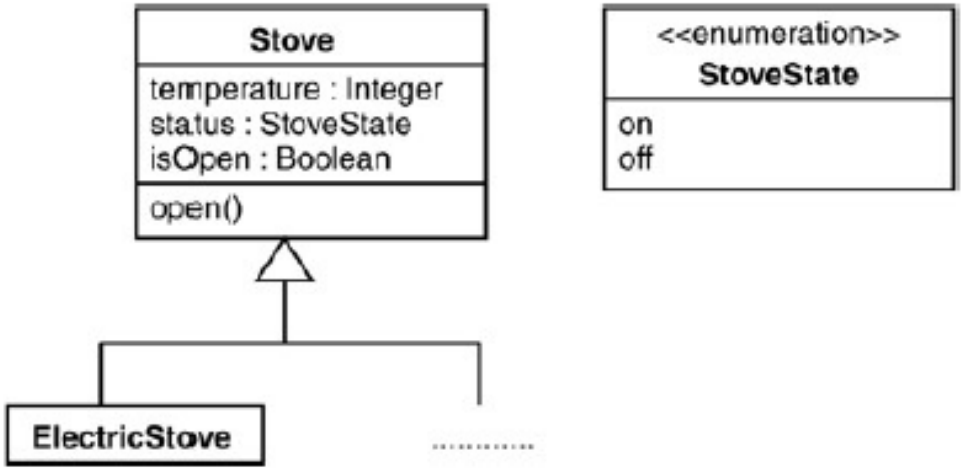
En nuestro ejemplo, se debería verificar que el invariante de la subclase implique el de la superclase, es decir:

```
temperature<=300 ⇒ temperature<=200
```

Lo cual no sucede porque la temperatura del horno electrico podría ser, por ejemplo, 210 que es menor que 300 pero no es menor que 200.

Herencia y Restricciones en OCL: Precondiciones y Postcondiciones

```
context Stove::open()
pre : status =
  StoveState::off
post: status =
  StoveState::off and
  isOpen
```




```
context
  ElectricStove::open()
pre : status =
  StoveState::off and
  temperature <= 100
post: isOpen
```

Una precondición puede debilitarse, no fortalecerse, en una redefinición de una operación en una subclase.

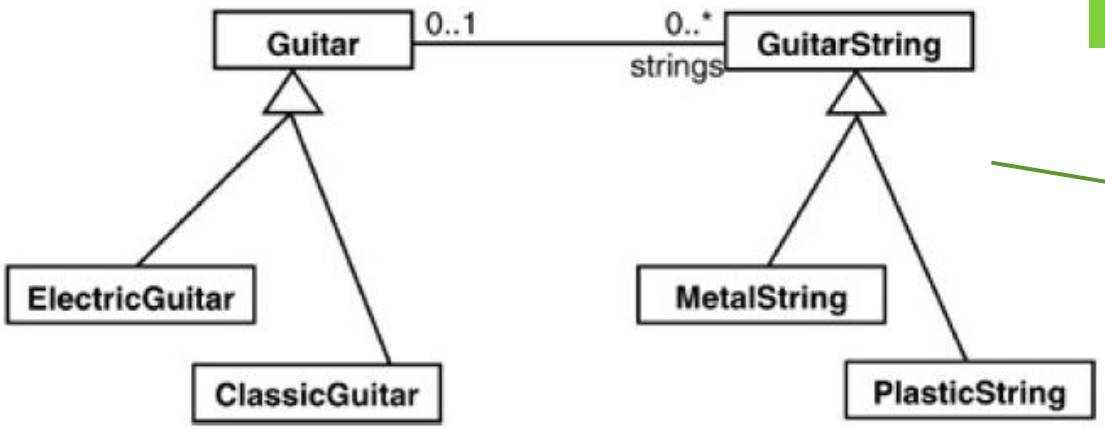
Una postcondición puede fortalecerse, no debilitarse, en una redefinición de una operación en una subclase.

Herencia
Algunas operaciones útiles sobre objetos

Operación	
`oclIsTypeOf(t:`*`Classifier`*`) : Boolean`	Da como resultado verdadero si el tipo del objeto y `t` son iguales. Ejemplos: **`context`** `Earning` **`inv`**`:self.oclIsTypeOf(Earning)-- true` **`Inv`**`:self.oclIsTypeOf(Company) -- false`
`oclIsKindOf(t:`*`Classifier`*`) : Boolean`	Determina si `t` es el mismo tipo o uno de los supertipos de un objeto. Ejemplos: **`context`** `Earning` **`inv`**`:self.oclIsKingOf(Transaction) -- true` **`inv`**`:self.oclIsKindOf(Earning) -- true`
`oclAsType(t:`*`Classifier`*`) :` *`instanceOfClassifier`*	Hace un cast explícito del objeto a un tipo `t` más especifico dentro de la jerarquía de clases. Devuelve `invalid` cuando el objeto real no es del tipo `t`.

Herencia
Uso de la operación **oclIsTypeOf**

EJEMPLO



El diagrama no modela las dos restricciones de abajo

- Una guitarra clásica solo puede tener cuerdas plásticas

```
context ClassicGuitar
inv strings1: strings->forAll(s |
  s.oclIsTypeOf(PlasticString) )
```

- Una guitarra electrica solo puede tener cuerdas metálicas

```
context ElectricGuitar
inv strings2: strings->forAll(s |
  s.oclIsTypeOf(MetalString) )
```

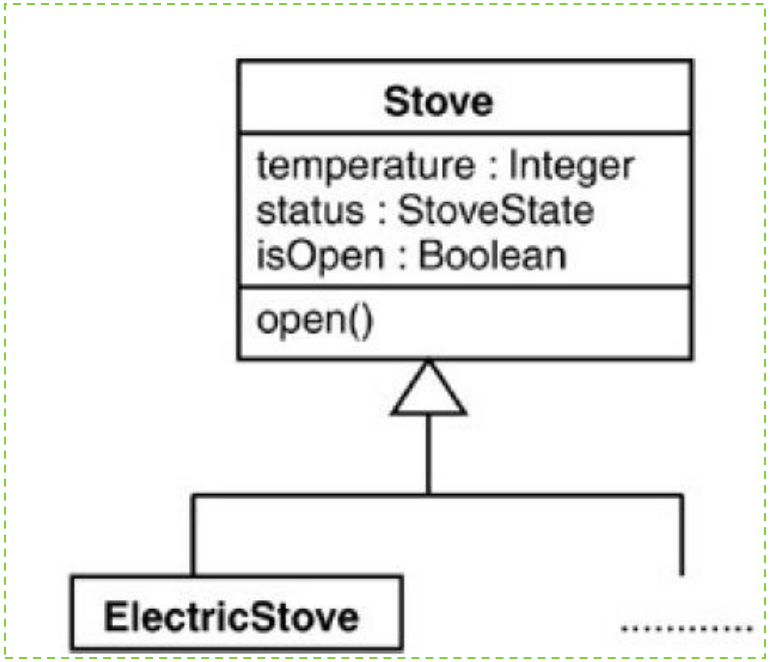
Teniendo en cuenta la Herencia : Uso de la operación **oclAsType**

EJEMPLO

- Permite acceder las propiedades redefinidas de la superclase desde la subclase.

```
context ElectricStove
   …
  self.open()
 -- accede a la operación open()
 -- definida en ElectricStove

 self.oclAsType(Stove).open()
 -- accede a la operación open()
 -- definida en Stove
```



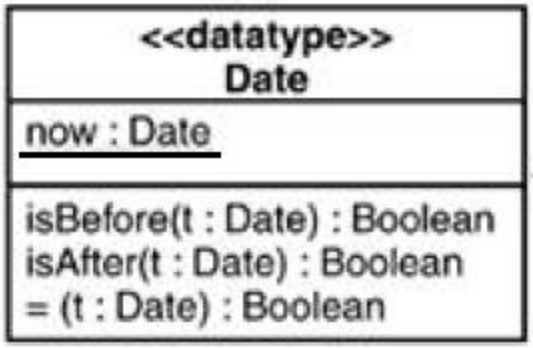
Uso de propiedades estáticas

- Para acceder a atributos y operaciones de clase

  NombredeClase::*atributo/operación*

- Ejemplo:

  Date::now



Operación allInstances()

- Devuelve todas las instancias de una clase
- Sintaxis

```
NombreClase.allInstances()
```

- Ejemplo:

```
-- No existen dos personas con el mismo
-- nombre
context Person
inv: Person.allInstances()->forAll(p1, p2
  | p1 <> p2 implies p1.name <> p2.name)
```

Algunos Consejos y Sugerencias

- Colocar <u>siempre</u> nombres de rol en los finales de asociación
- Evitar expresiones complejas
  - eligiendo bien el contexto
  - evitando el uso de `allInstances` toda vez que sea posible (recordar que un invariante en una clase siempre se aplica a todas las instancias de la clase)
  - dividiendo en varios invariantes los invariantes con múltiples `and`

```
context Classifier
inv:
...
inv:
```

  - usando la abreviatura **.** para `collect`

Bibliografía

- OMG (2014). Object Constraint Language Version 2.4 - http://www.omg.org/spec/OCL/2.4

- Jos Warmer, Anneke Kleppe (2003). The Object Constraint Language, Getting Your Models Ready for MDA, Second Edition. : Addison Wesley Publisher. August 29, 2003